DESIGN AND IMPLEMENTATION OF A FIDO2/U2F COMPLIANT AUTHENTICATION TOKEN FOR SECURE PASSWORDLESS AUTHENTICATION

A PROJECT REPORT

Submitted by

VAISHNAVI E (2116231901059)

AAKASH S (2116231901001)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

(CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this Project report titled **"Design and Implementation of a FIDO2/U2F Compliant Authentication Token for Secure Passwordless Authentication"** is the bonafide work of **"VAISHNAVI E (2116231901059), AAKASH S(2116231901001)"** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE	**SIGNATURE**
Dr. P. Tamilselvi, Ph.D.,	Mr. Benedict J.N., M.E.,
PROFESSOR & HEAD OF THE DEPARTMENT	**SUPERVISOR**
Department of Computer Science and Engineering (Cyber Security)	Assistant Professor (SG)
Rajalakshmi Engineering College	Department of Computer Science and Engineering (Cyber Security)
Chennai – 602 105	Rajalakshmi Engineering College
	Chennai – 602 105

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner **External Examiner**

ABSTRACT

Passwords are exposed to phishing, reuse and server breaches. FIDO2 replaces the shared secret with a key pair bound to the web site, and a hardware security key is the strongest place to hold the private key, but commercial keys are closed products. This project designed and implemented a FIDO2 authentication token on a low-cost ESP32-S3 development board. The firmware presents a USB HID interface with CTAPHID framing, implements the CTAP2 commands authenticatorGetInfo, authenticatorMakeCredential and authenticatorGetAssertion, signs with ECDSA P-256 and SHA-256 through mbedTLS, stores credentials in a dedicated flash partition and requires a button press as proof of user presence. A relying party built with FastAPI and py_webauthn issues single-use challenges and verifies every response.

On the ESP32-S3 at 240 MHz, a signature took 40.93 ms and key generation with a pairwise-consistency test 214.03 ms (n = 20). The complete firmware occupies 344,752 bytes and each credential needs a record of 155 bytes. The firmware sources passed more than 243,000 checks on the host, including 360,000 fuzzed inputs, without a failure or a crash, and replay, wrong-origin, unknown-credential, missing-presence and counter-regression cases were all rejected. Keys are stored in unencrypted flash and there is no PIN, so the token is a research prototype and not a certified product.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for the encouragement and guidance throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project, and our parents, friends, faculty members and supporting staff for their encouragement and support.

VAISHNAVI E

AAKASH S

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
1.	INTRODUCTION	1
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	3
	1.4 SCOPE	3
	1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS	4
	1.6 SDG ALIGNMENT	5
2.	LITERATURE REVIEW	6
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	6
	2.2 RESEARCH GAPS	9
	2.3 IMPLEMENTED SYSTEM	9
	2.3.1 Overview of the implemented system	9
	2.3.2 Objectives of the system	10
	2.3.3 Advantages of the system	10
3.	SYSTEM DESIGN	11
	3.1 SYSTEM ARCHITECTURE	11
	3.2 MODULE DESIGN	12
	3.3 DATA FLOW/UML DIAGRAMS	17

CHAPTER NO.	TITLE	PAGE NO.
	3.4 THREAT MODEL AND ATTACK SURFACE	20
	3.5 SECURITY CONTROLS	22
4.	SYSTEM DESCRIPTION	23
	4.1 DATA COLLECTION	23
	4.2 DATA PREPROCESSING	25
	4.2.1 Algorithms / Methodology	26
	4.2.2 Results and discussion	29
5.	CONCLUSION AND FUTURE ENHANCEMENTS	37
	5.1 CONCLUSION	37
	5.2 FUTURE ENHANCEMENTS	37
	REFERENCES	38
	APPENDIX I	40

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	Alignment of the project with the Sustainable Development Goals	5
2.1	Comparison of authentication approaches	8
3.1	Hardware and software used in the project	12
3.2	Threat model of the authentication token	21
3.3	Security controls and the requirements they address	22
4.1	Execution time of the cryptographic operations (n = 20)	30
4.2	Results of the software verification	33
4.3	Security test matrix	34
A1.1	Raw execution times of run 2 (optimised build), in microseconds	40
A1.2	Raw execution times of run 1 (unoptimised build, partial log), in microseconds	40

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	System architecture of the authentication token	11
3.2	Layered module design of the token firmware	13
3.3	Structure of the CTAPHID initialization and continuation packets	14
3.4	Data flow diagram of registration and authentication	17
3.5	Sequence diagram of credential registration	18
3.6	Sequence diagram of passwordless authentication	19
3.7	Activity diagram of request processing in the firmware	20
3.8	Trust boundaries and attack surface of the system	22
4.1	Layout of the authenticator data in registration and authentication	27
4.2	Mean execution time of the cryptographic operations	30
4.3	Execution time in the unoptimised and the optimised build	31
4.4	Size of the main components in the firmware image	32

LIST OF ABBREVIATIONS

AAGUID	Authenticator Attestation Globally Unique Identifier
CBOR	Concise Binary Object Representation
COSE	CBOR Object Signing and Encryption
CRC	Cyclic Redundancy Check
CTAP	Client to Authenticator Protocol
CTAPHID	CTAP over USB Human Interface Device
DER	Distinguished Encoding Rules
DRBG	Deterministic Random Bit Generator
ECDSA	Elliptic Curve Digital Signature Algorithm
ES256	ECDSA with the P-256 curve and SHA-256
FIDO	Fast Identity Online
GPIO	General-Purpose Input/Output
HID	Human Interface Device
NFC	Near Field Communication
NVS	Non-Volatile Storage
PIN	Personal Identification Number
RNG	Random Number Generator
RP	Relying Party
SDG	Sustainable Development Goal
SHA	Secure Hash Algorithm
U2F	Universal Second Factor
UP	User Present
USB	Universal Serial Bus
UV	User Verified

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Passwords are still the usual way in which a person proves identity to a web service. A password is a secret shared between the user and the server, and every weakness of password authentication follows from that fact. The user can be tricked into typing the secret into a fake page, the same secret is often reused on several services, and the server must keep a verifier that becomes a target when its database is breached. Bonneau et al. [1] compared thirty-five proposed replacements for passwords against twenty-five usability, deployability and security benefits and found that none of them kept every benefit of passwords, which explains why passwords have survived for so long.

The scale of the problem has been measured. Thomas et al. [2] studied credential theft between March 2016 and March 2017, with Google accounts as the case study, and identified 788,000 potential victims of keyloggers, 12.4 million potential victims of phishing kits and 1.9 billion usernames and passwords exposed through data breaches. Between 7 % and 25 % of the exposed passwords matched the current password of the victim's Google account. One-time passwords sent by SMS or generated by an application reduce the damage of a stolen password, but the code is still a value that the user types, so a phishing page can relay it to the real site within its validity period. For this reason the NIST digital identity guidelines classify out-of-band authentication over the public telephone network as a restricted authenticator [3].

The FIDO Alliance addressed the root cause by removing the shared secret. In the Universal Second Factor (U2F) protocol [4], and in its successor FIDO2, the user holds an authenticator that creates a separate public and private key pair for every web service. The service stores only the public key. During login the service sends a random challenge, the authenticator signs it with the

private key, and the service verifies the signature. The browser adds the origin of the page to the signed data, so a signature produced for a phishing site is useless at the real site. FIDO2 consists of two specifications: Web Authentication (WebAuthn) [5], the browser API published by the W3C, and the Client to Authenticator Protocol (CTAP) [6], which defines how the browser talks to an external authenticator over USB, NFC or Bluetooth.

A hardware security key is the form of authenticator that keeps the private key outside the computer. Commercial keys are sealed products, and their firmware, key storage and internal timing cannot be inspected by the people who rely on them. A general-purpose microcontroller with a native USB interface offers a different route: every layer, from the USB descriptors to the signature routine, can be written, measured and examined. This project followed that route and built a FIDO2 authentication token on an ESP32-S3 development board.

1.2 PROBLEM STATEMENT

Password-based login cannot resist phishing, credential reuse and server-side breaches, and one-time codes inherit the phishing weakness. FIDO2 security keys solve these problems, but the commercial products are closed, their cost limits wide distribution to students and small organisations, and their design cannot be studied or adapted. There is a need for an authenticator that implements the FIDO2 protocols on low-cost, widely available hardware, whose firmware is fully open to inspection, and whose security properties and costs are established by measurement and not by assumption.

The problem addressed in this project is therefore the design, implementation and evaluation of a USB hardware authentication token on the ESP32-S3 microcontroller that generates and protects per-site ECDSA key pairs, answers WebAuthn registration and authentication requests through CTAP2, requires a physical user action before it signs, and is tested against replay, wrong-origin and malformed-input attacks.

1.3 OBJECTIVES

The objectives of the project were the following.

1. To build a USB authenticator on an ESP32-S3 development board without a custom printed circuit board.
2. To implement the FIDO HID transport (CTAPHID) so that the host recognises the board as a FIDO device without a vendor driver.
3. To implement the CTAP2 commands authenticatorGetInfo, authenticatorMakeCredential and authenticatorGetAssertion.
4. To generate ECDSA P-256 key pairs on the device from a hardware entropy source and to keep every private key inside the device.
5. To store credentials in non-volatile memory so that they survive removal of power, and to detect corrupted records.
6. To require a button press as proof of user presence before any credential is created or used.
7. To build a WebAuthn relying party that issues single-use challenges and verifies every registration and authentication response.
8. To measure cryptographic execution time, firmware size, memory use and storage per credential, and to test the security behaviour of the token.

1.4 SCOPE

The project covers a roaming authenticator connected over USB. The firmware implements version FIDO_2_0 of CTAP2 with the ES256 algorithm (ECDSA over the NIST P-256 curve with SHA-256), packed self-attestation, and non-discoverable credentials that are selected through the allow list sent by the

relying party. The relying party is a local web application used for controlled and repeatable testing.

Several items are outside the scope. The older CTAP1/U2F message interface (CTAPHID_MSG) was not implemented; U2F is discussed in this report as the predecessor protocol whose security model FIDO2 retains, and the token announces to the host that it does not accept U2F messages. A PIN or biometric user verification, discoverable credentials, NFC and Bluetooth transports, a secure element, and FIDO Alliance certification are also outside the scope. The token is a research prototype, and the report states its limitations in Section 4.2.2.

1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

The following security requirements were fixed before the design began. Each requirement is traced to a control in Section 3.5 and to a test in Section 4.2.2.

- **SR1.** A private key is generated inside the token and is never returned.
- **SR2.** A credential is usable only for the relying party it was created for.
- **SR3.** Every signature covers a fresh challenge, so that it cannot be replayed.
- **SR4.** The token signs for a login only after a physical user action.
- **SR5.** A signature counter increases and is stored before each signature.
- **SR6.** Malformed input is rejected with an error and never causes a crash.
- **SR7.** Cryptography uses a vetted library, hardware entropy and self-tests.

- **SR8.** The relying party stores only public data and verifies every response.

Two considerations limit the claims made for the prototype. Keys are held in the ordinary flash memory of the microcontroller, so the token does not resist an attacker who can read the flash chip. There is no PIN, so possession of the token and a button press are sufficient to authenticate. Both points are treated as known limitations and not as solved problems.

1.6 SDG ALIGNMENT

The project supports four of the United Nations Sustainable Development Goals, as shown in Table 1.1.

Table 1.1 Alignment of the project with the Sustainable Development Goals

S. No.	SDG No. and goal	Relevant SDG target	Project contribution / alignment
1	SDG 9: Industry, innovation and infrastructure	Targets 9.1 and 9.5 (resilient infrastructure; research and technological capability)	A low-cost, open authenticator strengthens the security of digital infrastructure and builds local capability in embedded security design.
2	SDG 16: Peace, justice and strong institutions	Target 16.6 (effective and accountable institutions)	Phishing-resistant login protects the accounts of citizens and institutions from takeover and misuse.
3	SDG 8: Decent work and economic growth	Target 8.10 (access to financial services)	Strong authentication supports trustworthy online banking and payment services.
4	SDG 4: Quality education	Target 4.4 (technical and vocational skills)	Every layer of the token is open to study, so the design serves as a teaching platform for applied cryptography and secure firmware.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews the existing authentication methods and the published work on FIDO, states the research gaps and introduces the implemented system.

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

In password authentication the server stores a salted hash of the password and compares it with the hash of the value typed by the user. The approach needs no extra hardware and is understood by every user, which is why the comparative framework of Bonneau et al. [1] rates it highest for deployability. Its security depends on the secrecy of a value that the user must disclose at every login. The measurement study of Thomas et al. [2] showed that phishing kits and keyloggers collect not only the password but also the secondary information that services use for risk analysis, and that victims of phishing were far more likely to have their accounts hijacked than users whose passwords appeared only in a breach.

A second factor in the form of a one-time code limits the value of a stolen password. The code is delivered by SMS or computed from a shared seed and the current time. Both variants keep a secret on the server, and both ask the user to transfer a short value by hand. An attacker who operates a real-time phishing page can forward the code to the genuine site before it expires. Delivery over the telephone network adds the risks of SIM swapping and message interception, which led NIST to restrict its use [3].

U2F introduced the hardware security key as a second factor [4]. Lang et al. [7] describe the design and its deployment at Google. At registration the key creates a key pair for the site and returns the public key together with a key handle. At login the browser sends the challenge, the origin of the page and the

key handle, and the key returns a signature only after the user touches it. Because the origin is part of the signed data, a signature obtained by a phishing page does not verify at the real site. The authors report that security keys led to a higher level of security and user satisfaction and to lower support cost than one-time passwords. U2F remains a second factor: the user still types a password first.

FIDO2 generalises U2F so that the authenticator can replace the password. WebAuthn [5] defines the JavaScript interface through which a relying party creates a credential and requests an assertion, the structure of the authenticator data, and the verification procedure on the server. CTAP [6] defines the messages between the client platform and a roaming authenticator. Requests and responses are encoded in CBOR [8], public keys are expressed as COSE keys [9], and on USB the messages are carried in 64-byte HID reports by the CTAPHID framing. The authenticator signs the concatenation of the authenticator data and the SHA-256 hash of the client data, which contains the challenge and the origin. The standard signature algorithm is ES256, that is, ECDSA [10] on the P-256 curve with SHA-256.

Barbosa et al. [11] gave the first provable-security analysis of FIDO2. They proved that WebAuthn achieves the authentication property they defined, and showed that the PIN protocol of CTAP2 provides only a weaker guarantee, for which they proposed a stronger replacement. Bindel et al. [12] extended the analysis to CTAP 2.1 and WebAuthn Level 2 with a more detailed model and examined how the protocols can be instantiated with post-quantum algorithms. These results support the protocol design that the present project implements; they assume an authenticator that follows the specification, which is what an implementation has to demonstrate by testing.

Protocol security does not remove every risk. Ulqinaku et al. [13] observed that all the FIDO-supporting sites in the Alexa top 100 allowed the user to choose a weaker alternative to the security key. In their user study with a simulated real-time phishing page, 55 % of the participants fell for the attack and

a further 35 % would potentially have been susceptible in practice. The attack does not break the security key; it persuades the user not to use it.

Lyastani et al. [14] carried out a laboratory study with 94 participants that compared passwordless login with a FIDO2 security key against password login. Participants rated the security key as more usable and more acceptable, and their main concerns were losing the key and recovering the account afterwards. Reynolds et al. [15] ran two studies of the YubiKey. Setting the key up was difficult for many participants, whereas day-to-day use in a longer second study was rated favourably.

Commercial security keys keep the private key in a secure element and do not publish their firmware. Dauterman et al. [16] pointed out that the user must then trust the manufacturer completely, and designed True2F, a variant of U2F in which the browser contributes randomness to key generation and to signing, so that a faulty or malicious token cannot leak its keys. Roche et al. [17] showed that even a secure element is not immune to a determined attacker: by opening a Google Titan security key and recording the electromagnetic emission of its chip during signing, they recovered an ECDSA private key. The attack requires physical possession of the key and laboratory equipment.

Two open-source authenticators are widely known: Solo [18], based on the STM32L432 microcontroller, supports FIDO2 and U2F over USB and NFC, and OpenSK [19] is written in Rust for Nordic nRF52840 boards. Both show that an authenticator can be built in the open, on hardware families other than the ESP32-S3. Table 2.1 compares the approaches discussed in this section.

Table 2.1 Comparison of authentication approaches

Approach	Secret kept by the server	Resists phishing	Resists replay	Extra hardware
Password	Password verifier	No	No	None
One-time password	Verifier and seed	No	Partly	Phone
FIDO U2F key	Verifier and public key	Yes	Yes	Security key
FIDO2 key (this project)	Public key only	Yes	Yes	Security key

2.2 RESEARCH GAPS

The review leads to four observations about what the existing work does not provide.

- The usability studies and the formal analyses treat the authenticator as a black box. They do not describe how the transport, protocol, cryptographic and storage layers are built on a microcontroller.
- Commercial keys do not publish their firmware, so their handling of malformed input, power loss and counter updates cannot be examined by the user.
- Among the reviewed works, only True2F reports implementation measurements, and those concern a modified protocol on vendor hardware. The cost of each cryptographic step of standard FIDO2 on a general-purpose, low-cost microcontroller is not reported.
- The reviewed works do not offer a convention for separating results measured on hardware from results verified in software or derived by calculation, although the value of a prototype evaluation depends on that separation.

2.3 IMPLEMENTED SYSTEM

2.3.1 Overview of the implemented system

The implemented system is a USB authentication token built on an ESP32-S3-N16R8 development board, together with a relying party for testing. The token appears to the host as a FIDO HID device. Its firmware is divided into a hardware abstraction layer, the CTAPHID transport, a CTAP2 command processor and four services: a CBOR codec, a cryptographic wrapper around mbedTLS, a credential store on a dedicated flash partition, and a user-presence handler that reads the on-board button. The relying party is a web application

built with FastAPI and the py_webauthn library, with an SQLite database. A user registers the token once and afterwards logs in with a username and a button press, without a password.

2.3.2 Objectives of the system

- To implement the standard FIDO2 registration and authentication ceremonies on a low-cost, widely available microcontroller board.
- To keep the authenticator logic small and separate from the hardware layer, so that it can be compiled and tested on a personal computer as well as on the board.
- To verify the implementation with known-answer tests, an independent cryptographic library, and large-scale fuzzing of the transport and command layers.
- To report every result with its source: measured on hardware, produced by the build tools, verified in software, or derived.

2.3.3 Advantages of the system

- **Phishing resistance.** A credential is bound to the relying party identifier, and the signed data includes the origin recorded by the browser.
- **No reusable secret on the server.** The relying party holds public keys only, so a database breach does not allow impersonation.
- **Transparency.** The complete firmware is available for inspection, and the authenticator logic occupies only 12,539 bytes of the image.
- **Low cost and availability.** The token uses a standard development board and needs no custom hardware.
- **Measured behaviour.** Execution time, memory use and failure handling are documented with the method used to obtain them.

CHAPTER 3

SYSTEM DESIGN

This chapter presents the architecture of the system, the design of each module, the data flow and UML diagrams, the threat model and the security controls derived from it.

3.1 SYSTEM ARCHITECTURE

The system has three parts, shown in Figure 3.1. The client device is a computer with a web browser that supports WebAuthn. The relying party is the web application to which the user logs in. The authentication token is the ESP32-S3 board, connected to the client device by USB. The browser exchanges WebAuthn messages with the relying party over HTTPS and passes each request to the token through the CTAP layer of the operating system. The token never communicates with the relying party directly, and the relying party never sees anything from the token except public keys, authenticator data and signatures.

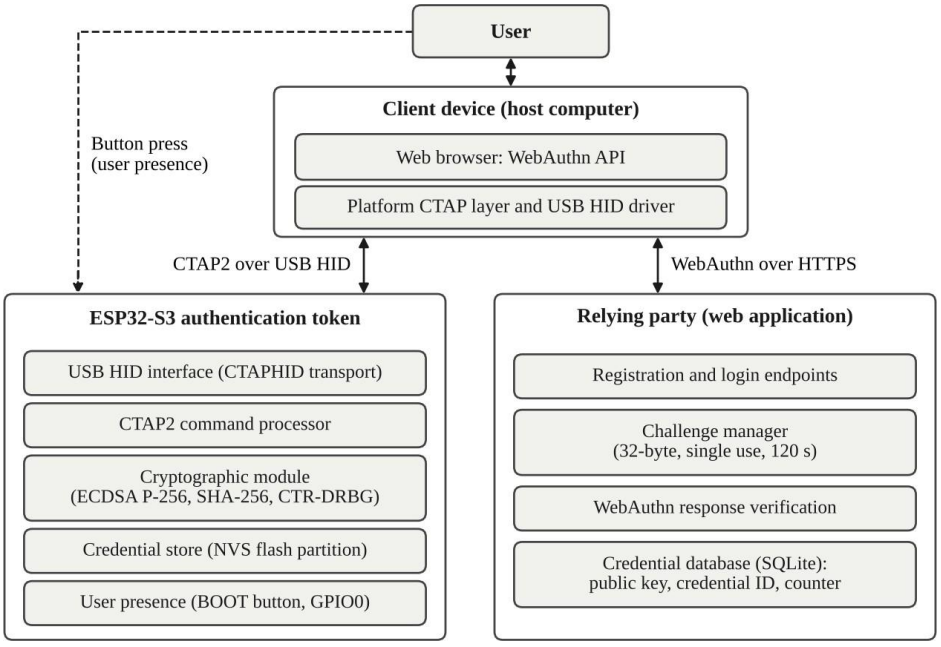


Figure 3.1 System architecture of the authentication token

The user takes part in two ways. The user starts registration or login in the browser, and the user presses the button on the token when it asks for confirmation. The second action reaches the token directly and cannot be generated by software on the host, which is the basis of the user-presence guarantee.

The hardware and software used for the prototype are listed in Table 3.1. The board carries two USB-C connectors. One is wired to a USB-to-UART bridge and is used for flashing and for the serial log. The other is wired to the native USB controller of the ESP32-S3 and carries the FIDO HID interface.

Table 3.1 Hardware and software used in the project

Component	Specification or version	Purpose
Development board	ESP32-S3-N16R8: dual-core Xtensa LX7, 16 MB flash, 8 MB PSRAM (disabled)	Runs the token firmware
Native USB port	USB-C; FIDO HID interface, VID 0x303A, PID 0x4004	Connection of the token to the host
USB-to-UART port	USB-C; WCH CH343 bridge	Flashing and serial log
Push button	On-board BOOT button on GPIO0, active low	User-presence input
Host computer	Windows PC with a WebAuthn-capable browser	Development, client and relying party
Firmware framework	ESP-IDF v5.3.6, mbedTLS 3.6.7, esp_tinyusb 1.7.6, esptool.py v4.12.0	Build, cryptography, USB and flashing [20]
Relying party	Python 3.11, FastAPI 0.141.1, py_webauthn 2.7.1, SQLite	Web application and verification
Test tools	python-fido2 1.2.0; pyca/cryptography, cbor2; GCC with sanitizers	Test client, cross-verification, fuzzing

3.2 MODULE DESIGN

The firmware is organised in layers, shown in Figure 3.2. The hardware abstraction layer is the only part that calls ESP-IDF, TinyUSB or the peripherals. The layers above it are written in portable C and receive the hardware functions through a small interface. This separation allowed the same

source files to be compiled for a personal computer and tested there with memory-error detection, as described in Section 4.1.

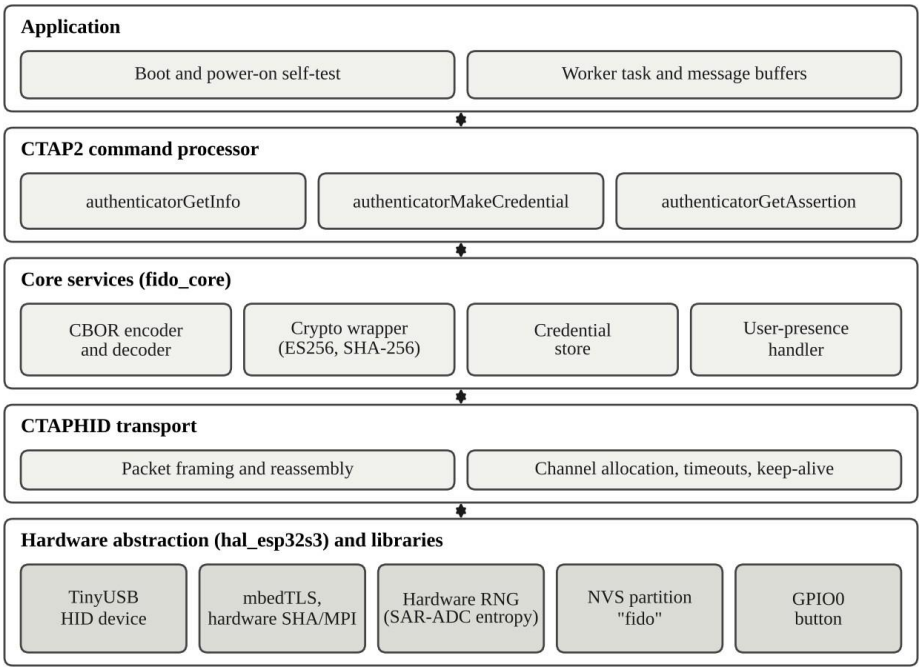


Figure 3.2 Layered module design of the token firmware

USB HID transport (CTAPHID). The token enumerates as a HID device with the FIDO usage page 0xF1D0 and usage 0x01. It exposes one interrupt IN and one interrupt OUT endpoint with 64-byte reports, no report identifier and a polling interval of 5 ms. The USB identifiers are the Espressif development values, VID 0x303A and PID 0x4004. Because the HID class is handled by the operating system, no driver has to be installed.

A CTAP message is usually longer than one report, so CTAPHID splits it into packets [6]. Figure 3.3 shows the two packet types. The initialization packet carries the channel identifier, the command with its top bit set, the payload length in two bytes and the first 57 bytes of payload. Each continuation packet carries the channel identifier, a sequence number and up to 59 bytes. The number of packets N needed for a payload of L bytes is given by Equation (3.1), and the largest payload is $57 + 128 \times 59 = 7609$ bytes.

$$N = 1 + \lceil (L - 57) / 59 \rceil, \quad \text{for } L > 57 \tag{3.1}$$

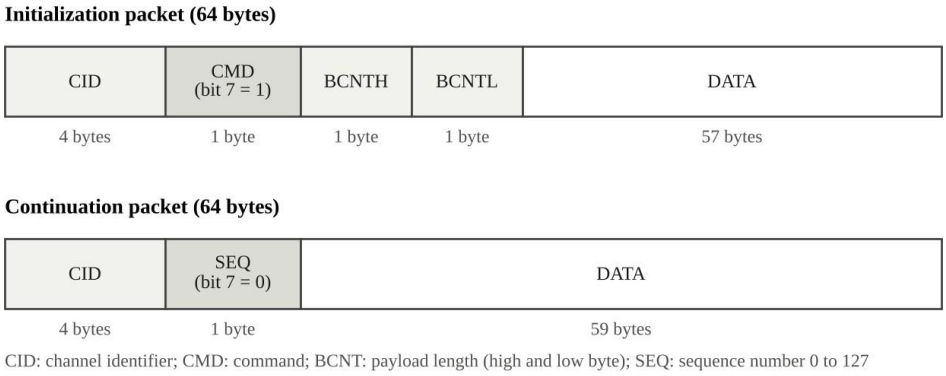


Figure 3.3 Structure of the CTAPHID initialization and continuation packets

The transport module allocates a channel when the host sends CTAPHID_INIT on the broadcast channel, echoes CTAPHID_PING, and passes the payload of CTAPHID_CBOR to the command processor. It reports the capability flags 0x0C, which state that CBOR messages are supported and that U2F messages (CTAPHID_MSG) are not. The module enforces the protocol rules: a packet from an unknown channel, a length above the maximum, a wrong sequence number or an unknown command produces a CTAPHID_ERROR response; a second channel that starts a message while another is in progress is told that the device is busy; and an incomplete message is discarded if the next packet does not arrive within 500 ms. While the token waits for the user it sends CTAPHID_KEEPALIVE packets, and it stops waiting when the host sends CTAPHID_CANCEL.

CTAP2 command processor. The command processor decodes the CBOR request, validates its parameters and calls the services. Three commands were implemented. The command authenticatorGetInfo returns the supported version FIDO_2_0, the authenticator identifier (AAGUID), the options rk = false, up = true and plat = false, and a maximum message size of 1200 bytes. The absence of the clientPin and uv options tells the browser that the token has no PIN and performs no user verification.

The command authenticatorMakeCredential creates a credential. The processor checks that ES256 is among the algorithms offered by the relying party, checks the exclude list so that the same token is not registered twice for one account, rejects options that are not supported, and then asks for user presence. The command authenticatorGetAssertion uses an existing credential. The processor searches the store for a record that matches both a credential identifier in the allow list and the SHA-256 hash of the relying party identifier. If the request sets the option up = false, the token answers without waiting for the button and leaves the user-present flag at zero; browsers use this form to find out which token holds a credential, and a relying party must not accept such an assertion as a login.

Cryptographic module. All cryptography is performed by mbedTLS; no elliptic-curve arithmetic was written for the project. The wrapper offers four operations to the rest of the firmware: random bytes, SHA-256, P-256 key generation and ES256 signing. Signatures are deterministic ECDSA as specified in RFC 6979 [21], with blinding of the scalar multiplication, and are returned in DER encoding. SHA-256 and the big-number arithmetic use the hardware accelerators of the ESP32-S3.

Random numbers come from a CTR-DRBG with AES-256 [22], seeded from the hardware random number generator. The radio subsystems of the chip are not used in this project, so the SAR-ADC entropy source is enabled explicitly, as the ESP-IDF documentation requires for true random output when Wi-Fi and Bluetooth are off [20]. The generator is reseeded before every key generation. After each key pair is generated, a pairwise-consistency test signs and verifies a test message with it, which confirms that the private and public parts belong together before the key is stored. At power-on the module runs known-answer tests of the hash and signature functions before the token accepts any request. The external PSRAM of the board is disabled, so that private keys are held only in the internal RAM of the chip while in use.

Credential store. Credentials are kept in a dedicated 256 KB partition of the flash memory, managed by the non-volatile storage (NVS) library of ESP-IDF, which provides wear levelling and atomic updates. Each credential is a fixed record of 155 bytes: a format version (1 byte), status flags (1), the credential identifier (16), the SHA-256 hash of the relying party identifier (32), the length of the user handle (1), the user handle (64), the P-256 private scalar (32), the signature counter (4) and a CRC-32 (4). The record is indexed by a 16-byte random credential identifier, which is the value the relying party stores and sends back in the allow list. A CRC-32 over the record detects corruption; a record that fails the check is never used and is counted as invalid. The firmware limits the store to 128 credentials. If a write to flash fails, the operation is aborted and an error is returned to the host.

User-presence handler. The on-board BOOT button, connected to GPIO0 and active low, serves as the user-presence input. GPIO0 is a strapping pin that the chip samples only at reset, so after boot the button is free for this purpose. When a command needs confirmation, the handler waits up to 30 seconds for a press. During the wait the transport sends keep-alive packets, which tell the host that the token is waiting for the user. A press sets the user-present flag. If the time expires, the token returns the status for a user-action timeout, and if the host cancels the request, it returns the status for a cancelled keep-alive. In both cases no signature is produced.

Relying party. The relying party is a FastAPI application. It issues a 32-byte random challenge for every ceremony; the challenge can be used once, expires after 120 seconds, and is bound to the browser session, the type of ceremony and the user. For registration it requests direct attestation, a cross-platform authenticator, no resident key and no user verification, with a timeout of 60 seconds. Responses are verified with py_webauthn. The application stores the credential identifier, the public key and the signature counter in SQLite, records every authentication event, and offers a dashboard for the user and an

administrative view that is restricted to authorised accounts. Because the credentials are not discoverable, login starts with the username.

3.3 DATA FLOW/UML DIAGRAMS

Figure 3.4 shows the movement of data. The relying party creates a challenge and keeps it for later comparison (process 1.0). The browser builds the client data, which contains the challenge and the origin, and passes its hash with the relying party identifier to the token (2.0 and 3.0). The token processes the command after the button press (4.0) and uses the credential store to create a key or to sign (5.0). The private key and the counter move only between the cryptographic process and the store D1. The response returns through the browser to the relying party, which verifies it against the stored challenge and the credential database D2 (6.0).

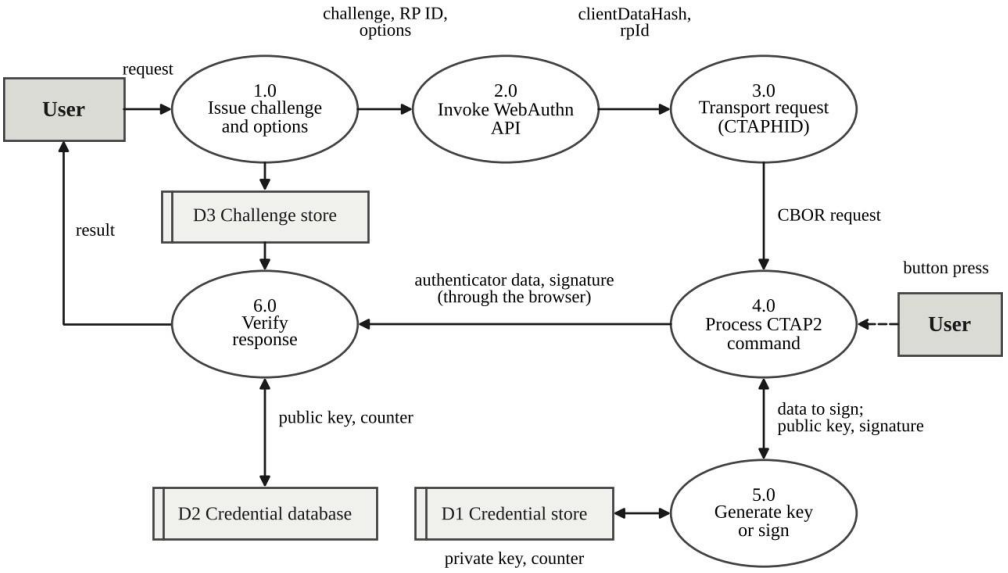


Figure 3.4 Data flow diagram of registration and authentication

Figure 3.5 shows the registration ceremony. The relying party sends the creation options with a 32-byte challenge. The browser calls navigator.credentials.create() and sends authenticatorMakeCredential to the token. After the button press the token generates the key pair, runs the pairwise

test, writes the record to flash, builds 148 bytes of authenticator data and signs the packed self-attestation with the new private key. The response is returned in five CTAPHID packets. The relying party verifies the challenge, the origin, the hash of the relying party identifier, the user-present flag, the algorithm and the attestation, and then stores the public key and the credential identifier.

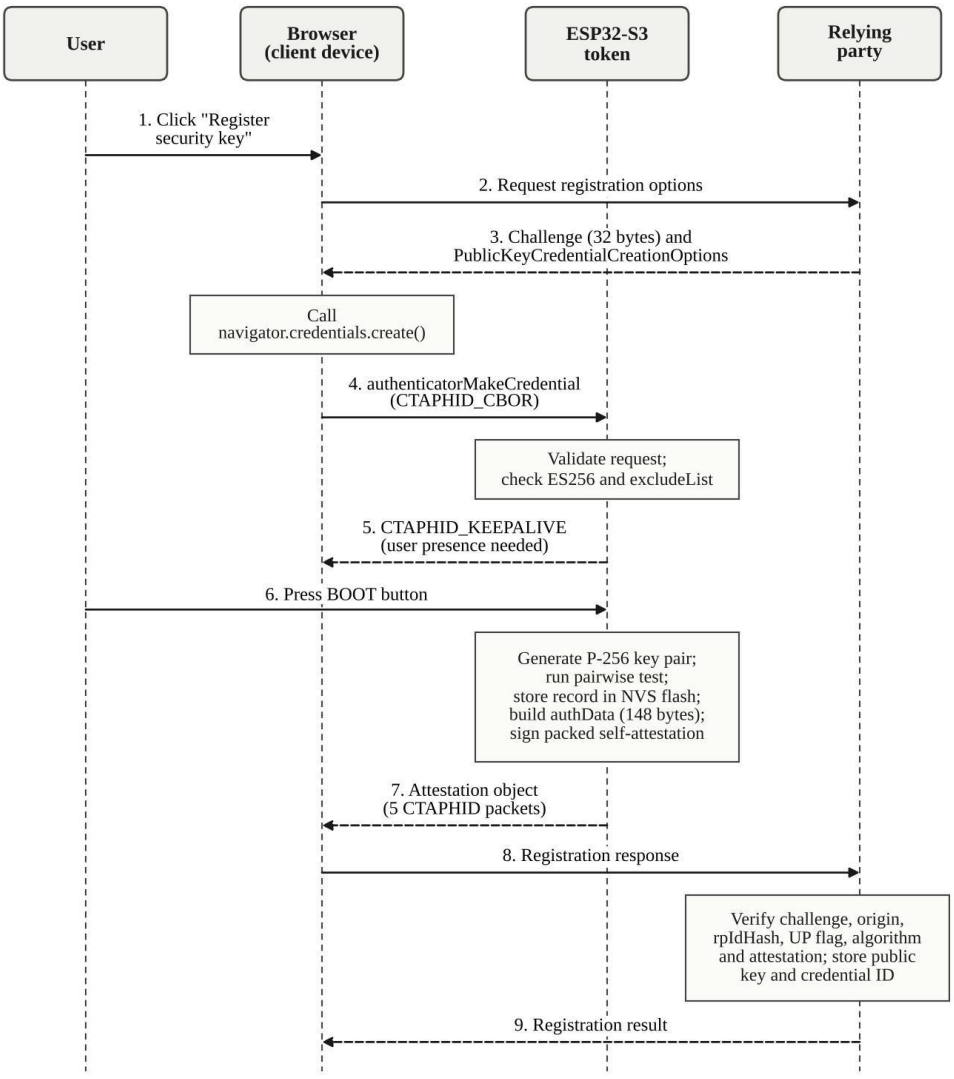


Figure 3.5 Sequence diagram of credential registration

Figure 3.6 shows the authentication ceremony. The relying party sends a fresh challenge, its identifier and the list of credential identifiers registered for the user. The browser calls navigator.credentials.get() and sends authenticatorGetAssertion. The token finds the record, waits for the button, increments the counter and stores it, builds 37 bytes of authenticator data and

signs the authenticator data followed by the client data hash. The response is returned in three packets. The relying party verifies the challenge, origin, relying party identifier, flags, counter and signature before it creates the session.

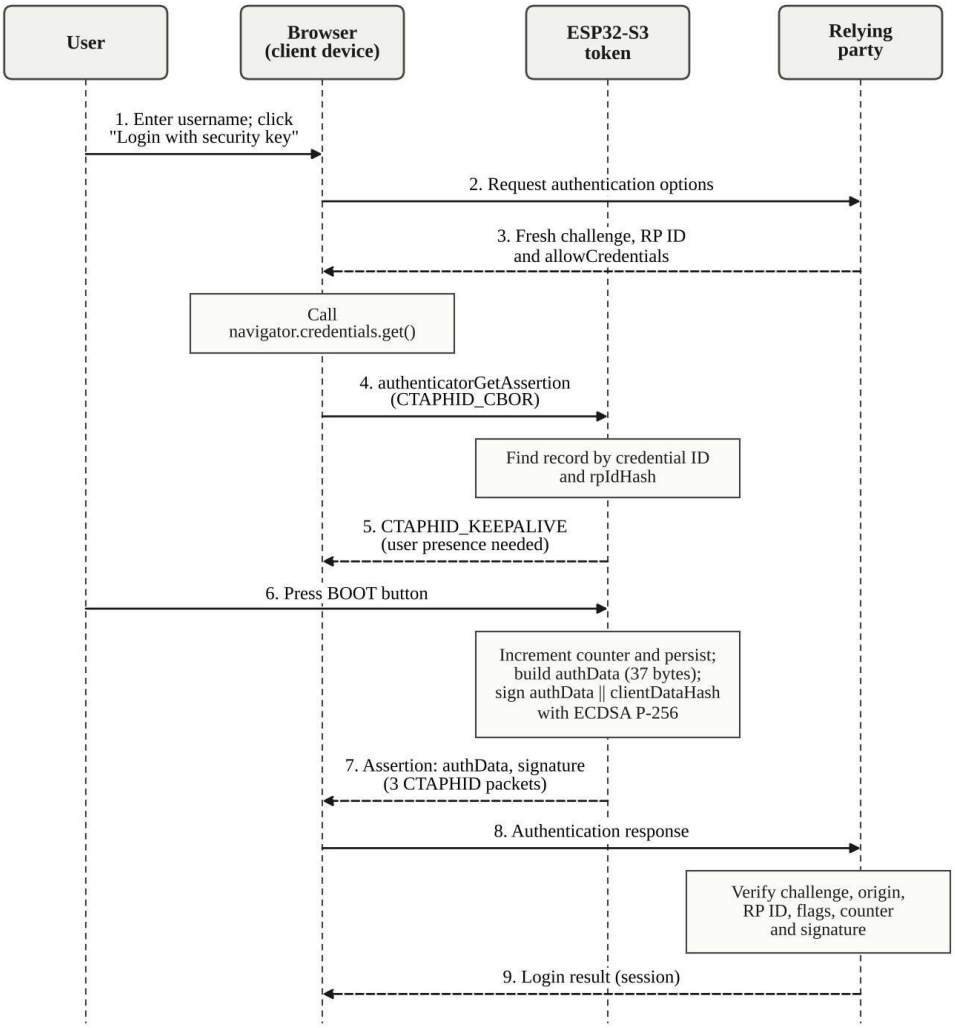


Figure 3.6 Sequence diagram of passwordless authentication

Figure 3.7 shows how the firmware handles one request. Errors are separated by layer: a framing error is answered by the transport with CTAPHID_ERROR, and every later failure is answered by the command processor with a CTAP2 status code. The key generation, the counter update and the signature are reached only on the path that passes through the button press.

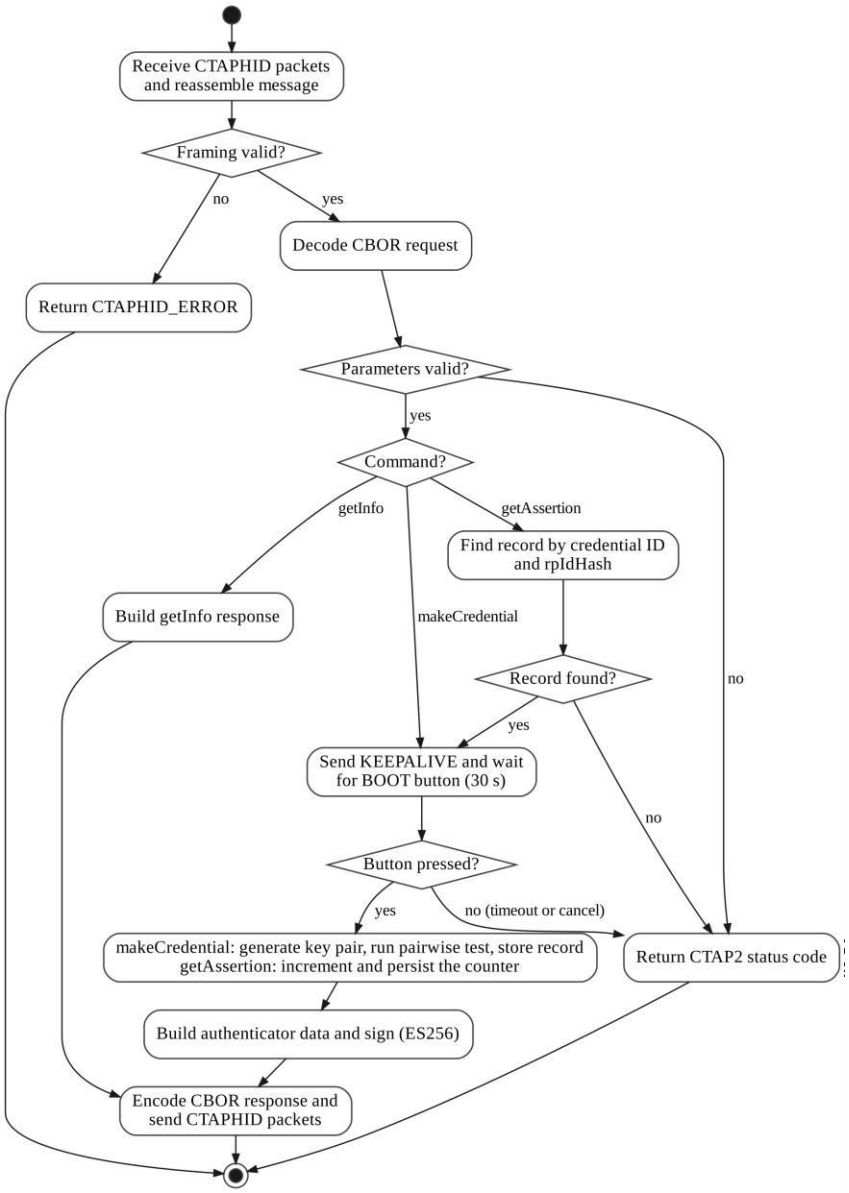


Figure 3.7 Activity diagram of request processing in the firmware

3.4 THREAT MODEL AND ATTACK SURFACE

The threat model assumes four kinds of attacker: a remote attacker who operates a phishing site or replays captured traffic; malicious software on the host computer, which can send any packet to the token but cannot press its button; an attacker who obtains the database of the relying party; and an attacker who gains physical possession of the token. Side-channel measurement, fault injection and attacks on the supply chain are outside the model.

Figure 3.8 shows the three trust zones and the six points at which an attacker can act. The token is trusted to keep the private keys. The host is not trusted with secrets and is treated as a possible source of hostile input. The relying party is trusted to verify responses but holds no secret of the user. Table 3.2 lists the threats, the entry point of each and the mitigation or the residual risk.

Table 3.2 Threat model of the authentication token

ID	Threat	Entry	Mitigation or residual risk
T1	A phishing site asks the token for an assertion	A3	The credential is looked up by the hash of the relying party identifier, so a different site finds no credential; the origin in the client data is checked by the relying party.
T2	A captured response is replayed	A3	Challenges are random, single use and expire after 120 s; the signature covers the hash of the client data.
T3	The database of the relying party is stolen	A4	Only public keys, credential identifiers and counters are stored; they do not allow a signature to be forged.
T4	Malformed or hostile USB packets are sent	A1	Length, sequence and channel checks, fixed buffers and timeouts; verified by fuzzing.
T5	Host malware requests signatures silently	A2	A button press is required; a request without it yields UP = 0 and is rejected by the relying party. Residual risk: a request issued while the user expects to press.
T6	A cloned token is used	A5	The counter is stored before signing; the relying party rejects a counter that does not increase. Detection needs both tokens to be used.
T7	The flash is read or the debug port is used	A5	Not mitigated in the prototype: keys are unencrypted in NVS; flash encryption, secure boot and JTAG disable are off.
T8	The token is lost or stolen	A6	Not mitigated in the prototype: there is no PIN, so possession and a button press suffice until the credential is removed at the relying party.
T9	A record is corrupted by power loss	Internal	CRC-32 on every record; invalid records are never used; a failed write aborts the operation.
T10	Random numbers are weak	Internal	CTR-DRBG seeded from the hardware generator with the SAR-ADC entropy source; reseed before each key generation; pairwise test.
T11	The firmware is replaced	A5	Not mitigated in the prototype: secure boot is off.

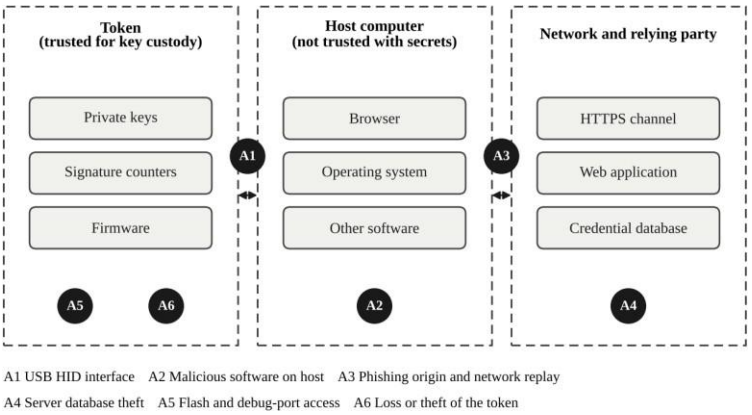


Figure 3.8 Trust boundaries and attack surface of the system

3.5 SECURITY CONTROLS

Table 3.3 lists the controls built into the system and the security requirement of Section 1.5 that each one serves. Controls C1 to C9 are in the token and control C10 is in the relying party.

Table 3.3 Security controls and the requirements they address

ID	Control	Implementation	Req.
C1	On-device key generation	P-256 keys are generated by mbedTLS from the CTR-DRBG; no command returns the private key.	SR1, SR7
C2	Relying party binding	rpIdHash is stored with the credential, compared at lookup and included in the signed authenticator data.	SR2
C3	Challenge binding	The signature covers authenticator data and client data hash; challenges are single use (120 s).	SR3, SR8
C4	User presence	Button press within 30 s; UP flag set only after the press; silent requests return UP = 0.	SR4
C5	Signature counter	A per-credential counter is incremented and written to flash before the signature is produced.	SR5
C6	Input validation	Length, sequence and channel checks; strict CBOR decoding; fixed buffers; 500 ms timeout.	SR6
C7	Record integrity	CRC-32 on each record; invalid records are ignored; a failed write aborts the command.	SR5, SR6
C8	Cryptographic self-tests	Known-answer tests at power-on; pairwise-consistency test after each key generation.	SR7
C9	Build hardening	Stack protector and memory protection on; PSRAM off so that keys stay in internal RAM.	SR1, SR6
C10	Server-side verification	py_webauthn verifies each response; the application checks the challenge state, the counter and the account.	SR8

CHAPTER 4

SYSTEM DESCRIPTION

This chapter describes how the data of the project were collected and prepared, the algorithms executed by the token and the relying party, and the results with their discussion.

4.1 DATA COLLECTION

The project does not use an external data set. Its data are the measurements and the test outcomes of the token, and they were collected from three sources: the development board, the firmware build tools, and test programs run on a personal computer.

The token was implemented on the ESP32-S3-N16R8 development board listed in Table 3.1. The hardware record of the board was read with esptool.py and from the device list of the host on 27 September 2026: silicon revision v0.2, a 40 MHz crystal, 16 MB of quad-SPI flash operated in DIO mode at 80 MHz, and 8 MB of embedded PSRAM, which the firmware leaves disabled. The Wi-Fi and Bluetooth radios of the chip are not used.

Two firmware images were produced from the same source tree. The benchmark firmware contains the cryptographic module and the measurement loop; it was built with ESP-IDF v5.3.6 (mbedTLS 3.6.7), flashed to the board and used for the timing measurements. The complete token firmware contains all the modules of Figure 3.2; its size and memory figures come from the build with ESP-IDF v5.3.2 (mbedTLS 3.6.2) in the continuous-integration environment.

The benchmark firmware was built twice. Version 0.1.0 (run 1) used the -Og optimisation level, a CPU frequency of 160 MHz, no fixed-point tables and prediction resistance on every call of the random bit generator. Version 0.1.1 (run 2) used -O2, 240 MHz, fixed-point tables and a reseed before each key

generation. Both builds used deterministic ECDSA, the NIST-optimised reduction for P-256 and the hardware SHA and big-number accelerators. The strong stack protector and memory protection were on; flash encryption, secure boot and JTAG disable were off. The partition table holds the factory application of 2 MB at 0x10000 and the credential partition of 256 KB at 0x210000.

Every result collected in this way is labelled with the kind of evidence behind it. Four categories are used, and only the first is reported as a measurement of the physical token.

- **Hardware-measured:** measured on the ESP32-S3-N16R8 board.
- **Build:** output of the firmware build tools, such as the image size and the memory map, stated with the ESP-IDF version.
- **Software-verified:** obtained from the C sources of the firmware compiled for a personal computer and tested there with a simulated button. These results verify the implementation and are not hardware results.
- **Derived:** calculated from measured values by a stated formula and reported as an estimate.

Testing was carried out at five levels. Unit tests exercised the cryptographic wrapper with published known-answer vectors and with invalid inputs. Transport tests drove the CTAPHID module with valid, malformed and random packets. Command tests covered the CTAP2 processor, the CBOR codec and the credential store, including corrupted records and failed writes. End-to-end tests connected the firmware code, through the python-fido2 client, to the relying party with py_webauthn. These four levels were run on the host under AddressSanitizer and UndefinedBehaviorSanitizer, so that any out-of-bounds access or undefined operation would stop the test. Signatures and encodings were checked a second time with OpenSSL through the pyca/cryptography library, which shares no code with mbedTLS. The fifth level is the physical board, on

which the firmware was flashed, the power-on self-test was run and the cryptographic operations were timed.

Execution times on the board were taken with the microsecond timer esp_timer_get_time(). The workload had the shape of a WebAuthn assertion: 37 bytes of authenticator data followed by a 32-byte client data hash. Each operation was repeated 20 times in the optimised build. Key generation was timed together with the reseed and the pairwise-consistency test, and the pairwise test was also timed alone. Seven metrics were defined for the evaluation: signature time (M1), key generation time (M2), credential creation time (M3) and authentication time (M4) inside the token, excluding the wait for the button; firmware size (M5); RAM usage (M6); and storage per credential (M7).

4.2 DATA PREPROCESSING

The collected data were prepared before they were reported. The timer values were recorded in microseconds for every iteration and are reproduced without change in Tables A1.1 and A1.2 of Appendix I. For each operation the mean, the standard deviation, the median, the minimum, the maximum and the 95 % confidence interval of the mean were computed over the 20 iterations of the optimised build and expressed in milliseconds. The time of key generation alone was not measured directly; it was derived for every iteration by subtracting the time of the pairwise-consistency test from the combined time. The log of the unoptimised build was recovered only in part, so its four available iterations were used for the comparison of the two builds and for nothing else. Derived values are marked as estimates wherever they appear.

The token also prepares the data of every request before an algorithm of Section 4.2.1 is executed. The transport reassembles the 64-byte CTAPHID packets into one message and checks the channel, the length and the sequence numbers. The command processor decodes the CBOR map, rejects a request in which a required parameter is missing or malformed, and computes the SHA-256 hash of the relying party identifier. The client data are hashed by the

browser, so the token receives a 32-byte client data hash and not the challenge or the origin themselves.

4.2.1 Algorithms / Methodology

Key generation and signature. A credential key pair consists of a private scalar d, chosen at random between 1 and $n - 1$, and the public point Q of Equation (4.1), where G is the base point of the P-256 curve and n is its order.

$$Q = d \cdot G \tag{4.1}$$

To sign a message m, the token computes e from SHA-256(m), derives the per-signature value k from d and e by the deterministic procedure of RFC 6979 [21], and computes the pair (r, s) of Equations (4.2) and (4.3), where x_1 is the x-coordinate of $k \cdot G$.

$$r = x_1 \bmod n \tag{4.2}$$

$$s = k^{-1} (e + r \cdot d) \bmod n \tag{4.3}$$

The verifier computes $u_1 = e \cdot s^{-1} \bmod n$ and $u_2 = r \cdot s^{-1} \bmod n$ and accepts the signature if the x-coordinate of the point in Equation (4.4) equals r modulo n [10]. Signing needs one multiplication of the fixed point G; verification needs in addition a multiplication of the variable point Q, which matters for the timing results in Section 4.2.2.

$$(x_1, y_1) = u_1 \cdot G + u_2 \cdot Q \tag{4.4}$$

Credential creation. On receiving authenticatorMakeCredential, the token performs the following steps.

1. Decode the request and check that it contains the client data hash, the relying party, the user and the list of acceptable algorithms.
2. Check that ES256 is in the list, that no credential of the exclude list is present on the token for this relying party, and that no unsupported option is requested.

3. Wait for the BOOT button for up to 30 seconds.
4. Reseed the random bit generator, generate the key pair (d, Q) and run the pairwise-consistency test.
5. Generate a 16-byte random credential identifier and write the credential record to flash.
6. Compute the hash of the relying party identifier, Equation (4.5), and build the authenticator data of Equations (4.6) and (4.7) with the flags UP and AT set (0x41).
7. Sign the authenticator data followed by the client data hash with d, Equation (4.8), and return the packed attestation object.

$$\text{rpIdHash} = \text{SHA-256}(\text{rpId}) \tag{4.5}$$

$$\text{authData} = \text{rpIdHash} \parallel \text{flags} \parallel \text{signCount} \parallel \text{attCredData} \tag{4.6}$$

$$\text{attCredData} = \text{AAGUID} \parallel L \parallel \text{credentialId} \parallel \text{COSE_Key}(Q) \tag{4.7}$$

$$\sigma = \text{ECDSA_Sign}(d, \text{authData} \parallel \text{clientDataHash}) \tag{4.8}$$

The symbol $\parallel$ denotes concatenation, attCredData is the attested credential data and L is the two-byte length of the credential identifier. Figure 4.1 shows the layout of the authenticator data for both commands.

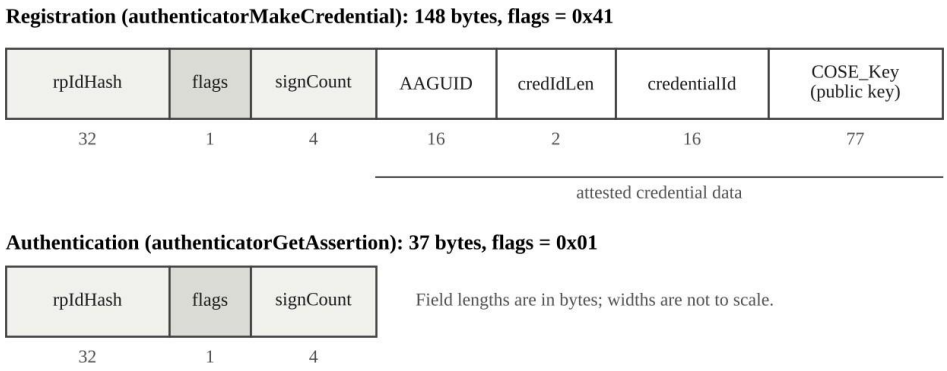


Figure 4.1 Layout of the authenticator data in registration and authentication

The token uses self-attestation: the attestation signature is made with the private key of the new credential itself, and no attestation certificate is sent. The relying party can therefore confirm that the response is well formed and that the sender holds the private key, but it learns nothing about the make of the authenticator.

Assertion generation. On receiving authenticatorGetAssertion, the token performs the following steps.

1. Decode the request and compute the hash of the relying party identifier.
2. For each credential identifier in the allow list, read the record and accept it only if its CRC-32 is correct and its stored rpIdHash equals the computed hash. If no record is accepted, return the status for no credentials.
3. Wait for the BOOT button for up to 30 seconds, unless the request is a silent probe with up = false.
4. Increment the signature counter of the record and write it to flash.
5. Build the authenticator data of Equation (4.9) with the flag UP set (0x01), or with flags 0x00 for a silent probe.
6. Sign with the private key of the record as in Equation (4.8) and return the credential identifier, the authenticator data and the signature.

$$\text{authData} = \text{rpIdHash} \parallel \text{flags} \parallel \text{signCount} \tag{4.9}$$

The counter is written before the signature is computed. If power is lost between the two steps, the counter has advanced without a signature being released, which is harmless; the opposite order could release two signatures with the same counter value.

Verification by the relying party. The browser sends the response to the relying party together with the client data, a JSON object that contains the

type of ceremony, the challenge and the origin of the page. The relying party computes the client data hash by Equation (4.10) and accepts the response only if all the following conditions hold.

$$\text{clientDataHash} = \text{SHA-256}(\text{clientDataJSON}) \tag{4.10}$$

- The challenge in the client data equals a challenge that was issued for this session, ceremony and user, has not been used and has not expired.
- The origin in the client data is the origin of the relying party.
- The rpIdHash in the authenticator data is the SHA-256 hash of the relying party identifier.
- The user-present flag is set.
- For registration, the algorithm is ES256 and the attestation signature verifies with the public key in the response.
- For authentication, the credential identifier belongs to the user, the signature verifies with the stored public key over the authenticator data and the client data hash, and the counter is greater than the stored value.

After a successful authentication the stored counter is replaced by the received value. Each failed condition has its own error code in the application, which made the outcome of the negative tests in Section 4.2.2 unambiguous.

4.2.2 Results and discussion

Cryptographic performance. Table 4.1 and Figure 4.2 give the execution times of the cryptographic operations on the board at 240 MHz (hardware-measured, run 2, n = 20). A signature took 40.93 ms on average. Key generation with the pairwise-consistency test took 214.03 ms, of which the test accounted for 178.40 ms, or 83 %; key generation alone took 35.63 ms. Verification took 137.50 ms and is therefore about 3.4 times slower than signing. The power-on self-test, measured once, took 316.06 ms.

Table 4.1 Execution time of the cryptographic operations (n = 20)

Operation	Mean (ms)	SD (ms)	Median (ms)	Min (ms)	Max (ms)	95 % CI (ms)
Key generation with pairwise test	214.030	0.106	214.018	213.836	214.243	±0.050
Key generation only (derived)	35.632	0.072	35.620	35.474	35.751	±0.034
Pairwise-consistency test	178.399	0.099	178.404	178.180	178.632	±0.046
ES256 sign	40.927	0.047	40.926	40.844	41.018	±0.022
ES256 verify	137.496	0.073	137.487	137.383	137.655	±0.034

The timing is nearly constant. The standard deviation is at most 0.2 % of the mean for every operation, and the difference between the slowest and the fastest of the 20 runs is at most 0.45 ms. The pairwise test consists of one signature and one verification, and its mean of 178.40 ms agrees with the sum of the separately measured signing and verification times, 178.42 ms, which confirms that the measurements are consistent with each other.

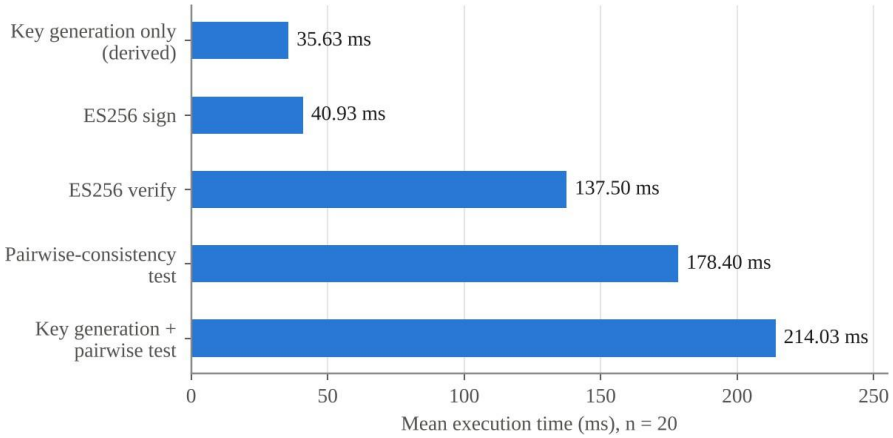


Figure 4.2 Mean execution time of the cryptographic operations

Effect of the build configuration. Figure 4.3 compares the unoptimised build (run 1) with the optimised build (run 2), whose settings are given in Section 4.1. Key generation with the pairwise test fell from 672.10 ms (range 655.82 to 720.61) to 214.03 ms, signing from 169.95 ms (169.80 to 170.05) to 40.93 ms and verification from 327.46 ms (327.21 to 327.64) to 137.50 ms. The speed-up factors are 3.14, 4.15 and 2.38.

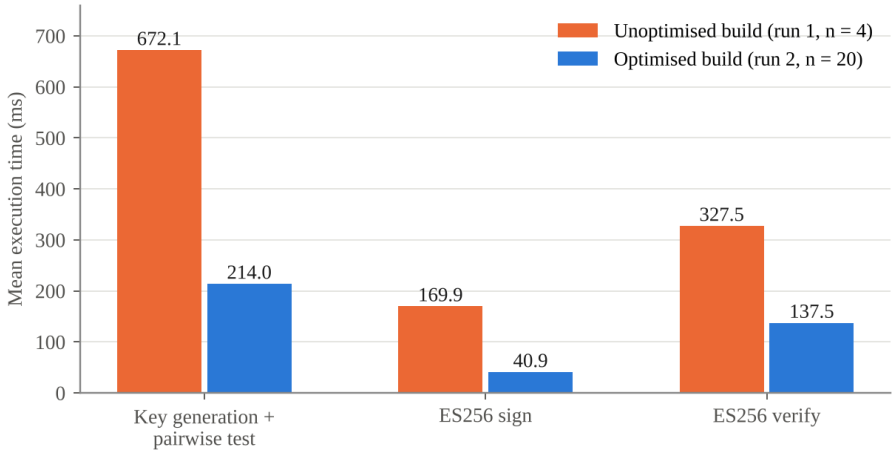


Figure 4.3 Execution time in the unoptimised and the optimised build

Two cautions apply to this comparison. Four settings were changed together, so the gain cannot be attributed to any one of them. The log of run 1 was recovered only in part, so its means rest on four iterations. The comparison is reported as the difference between an unoptimised and an optimised configuration and nothing more. In run 1 the measurement loop did not yield to the scheduler for about 24 seconds and the task watchdog issued a warning without resetting the chip; the loop was corrected in version 0.1.1.

Message sizes and resource usage. The sizes of the messages produced by the firmware code are fixed by the protocol encoding and do not depend on the platform (software-verified). The getInfo response is 51 bytes. A registration response is 244 to 246 bytes and is carried in five CTAPHID packets; an authentication response is 152 to 155 bytes and is carried in three, in agreement with Equation (3.1). The COSE key of a P-256 public key is 77 bytes. The variation in the response sizes comes from the DER encoding of the signature, whose length depends on the leading bits of r and s: among 230 signatures, one was 69 bytes long, 52 were 70 bytes, 132 were 71 bytes and 45 were 72 bytes, with a mean of 70.96 bytes.

The complete token firmware occupies 344,752 bytes, which is 16.4 % of the 2 MB application partition (build, ESP-IDF v5.3.2); the benchmark firmware occupies 285,072 bytes. Static data in internal RAM amount to 73,983

of 341,760 bytes, or 21.65 % (build). At the end of boot the benchmark firmware reported 373,532 bytes of free heap on the board (hardware-measured). A credential record of 155 bytes takes about seven NVS entries of 32 bytes, that is, about 224 bytes of flash (derived estimate), and the credential partition reported 8,064 free entries at first boot (hardware-measured).

Figure 4.4 breaks the image down by component. The cryptographic library is the largest of the listed components at 66,145 bytes, about 29 KB of which are the fixed-point tables that produced part of the speed-up reported above. The main component takes 17,463 bytes, of which 15,309 bytes are the CTAPHID request and response buffers. The authenticator logic of the project (CTAPHID, CTAP2, CBOR, credential store and cryptographic wrapper) occupies 12,539 bytes, which is 3.6 % of the image. The remainder of the image belongs to other components of ESP-IDF.

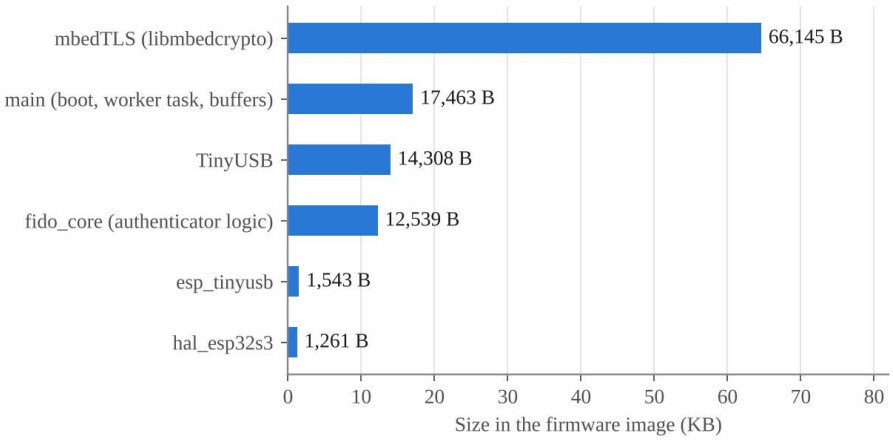


Figure 4.4 Size of the main components in the firmware image

Evaluation metrics. The seven metrics defined in Section 4.1 take the following values. The signature time (M1) is 40.93 ms and the key generation time with the pairwise test (M2) is 214.03 ms, both hardware-measured. The credential creation time (M3) and the authentication time (M4) inside the token are lower bounds derived from the measured primitives: a registration needs one key generation with pairwise test and one signature, at least 214.0 + 40.9 = 255 ms, and an authentication needs one signature, at least 41 ms. These bounds

exclude CBOR processing, hashing, the flash write and the USB transfer. The firmware size (M5) is 344,752 bytes, the RAM usage (M6) is 73,983 bytes of static data with 373,532 bytes of free heap at boot in the benchmark firmware, and the storage per credential (M7) is a 155-byte record, or about 224 bytes in NVS.

Software verification. Table 4.2 summarises the verification of the firmware sources on the host. More than 243,000 individual checks were executed without a failure, and 360,000 random or fuzzed packets and requests were processed without a crash or a sanitizer report. The cryptographic output was confirmed by OpenSSL for 50 signatures, and the RFC 6979 known-answer test reproduced the published signature bytes exactly.

Table 4.2 Results of the software verification

Suite	Content	Result
Cryptographic unit tests	Known-answer tests (SHA-256, RFC 6979 P-256); invalid keys and arguments; altered messages and signatures; off-curve keys; 200 unique key pairs	21,740 checks, 0 failures
Independent cross-verification	50 signatures verified by OpenSSL; strict DER; canonical CBOR of the COSE key	50 of 50 verified
CTAPHID transport	INIT and PING from 0 to 7609 bytes; all error codes; sequence errors; busy channel; timeouts; CANCEL; KEEPALIVE; 300,000 random packets	161,357 checks, 0 failures, no crash
CTAP2, storage and CBOR	Ceremonies; relying party binding; counter; user-presence timeout and cancel; exclude list; unsupported options; corrupted records; failed write; 60,000 fuzzed requests	60,034 checks, 0 failures, no crash
End-to-end	Firmware code, python-fido2 client and relying party with py_webauthn	20 of 20 security checks passed
Relying party and web interface	Access control, export and security headers; register and login in headless Chromium with a virtual authenticator	10 of 10 tests passed; interface test passed

Security test results. Table 4.3 lists the security tests with the expected behaviour and the result. On the board, the firmware booted and the power-on self-test passed. The protocol tests were run in software verification, with the firmware code connected to the relying party through the python-fido2 client. Every negative case was rejected for the intended reason: a replayed or

altered challenge by the relying party, a request for another relying party and an unknown credential by the token, a missing button press by a timeout without a signature, and a counter lower than the stored value by the relying party.

Table 4.3 Security test matrix

ID	Test	Expected result	Result
T02	Flash and boot on the board	Boots; self-test passes	PASS (hardware)
T03	Valid packets: INIT, PING up to 7609 bytes, getInfo	Correct echo or response	PASS
T04	Malformed packets: bad channel, excessive length, bad sequence, short INIT, unknown command, empty CBOR, message timeout	Correct error; device stays responsive	PASS (11 cases and 300,000 random packets)
T05	Credential generation	Valid packed attestation; record stored	PASS
T07	Registration on the relying party	Accepted; public key stored	PASS
T08	Authentication	Assertion accepted	PASS (3 of 3)
T09	Replayed, stale or altered challenge	Rejected	PASS (challenge not found; challenge mismatch)
T10	Wrong origin or wrong relying party	Rejected	PASS (origin mismatch at relying party; no credentials at token)
T11	Unknown credential	Error	PASS (no credentials at token; unknown credential at relying party)
T-UP	No button press	Timeout; no signature	PASS (user-action timeout; cancel handled)
T-UPF	Silent assertion (up = false) used as a login	Rejected by relying party	PASS (UP flag 0; rejected)
T-CNT	Counter strictly increasing	Monotonic	PASS
T-CLONE	Assertion with a counter below the stored value	Rejected	PASS (counter regression)
TH12	Another session adds a key to an existing account	Refused	PASS (HTTP 403)
T-CORRUPT	Corrupted credential record	Record never used	PASS (reported as no credentials)

Discussion. The measurements show that the ESP32-S3 is fast enough for the purpose. The cryptographic work of a login is one signature of about 41 ms, and that of a registration about 255 ms. Both are short beside the time a person takes to press a button.

The pairwise-consistency test is the most expensive step, at 83 % of the key generation time. It was kept because it is paid once per registration and prevents a faulty key pair from being stored. The gap between signing and verification follows from Section 4.2.1: the fixed-point tables accelerate multiplications of the base point, which is all that signing and key generation need, whereas verification also multiplies the variable public point. The token verifies only inside the pairwise test, so the slower operation does not affect login.

The very small spread of the execution times is a necessary property of a careful implementation, but it is not evidence of resistance to side-channel analysis, which was not evaluated.

In terms of resources the authenticator is small: the project code is less than one fifth of the size of the cryptographic library it calls. The two message buffers of 7,609 bytes each make up most of the RAM used by the main component; since the token advertises a maximum message size of 1200 bytes, they could be reduced in a later version.

The verification results give confidence in the correctness of the firmware sources. The transport and command layers processed 360,000 random and fuzzed inputs under two sanitizers without a memory error, and the negative tests failed for the intended reasons and at the intended layer. The authenticator logic verified on the host is the same source code that is built into the firmware.

Limitations. The prototype has the following limitations.

- Private keys are stored unencrypted in flash (NVS), and flash encryption, secure boot and JTAG disable are off, so an attacker with physical access can extract the keys or replace the firmware.
- There is no PIN or other user verification. A stolen token and a button press are sufficient to authenticate.
- Credentials are non-discoverable only, and the CTAP1/U2F message interface is not implemented.
- The token is not FIDO certified: attestation is self-attestation, the AAGUID is not registered in the FIDO Metadata Service, and the USB identifiers are development values.
- The hardware measurements cover boot, self-test and the cryptographic operations; the protocol behaviour was verified in software, and browser ceremonies on the board, side-channel analysis and fault injection were not evaluated.

CHAPTER 5

CONCLUSION AND FUTURE ENHANCEMENTS

5.1 CONCLUSION

This project designed and implemented a FIDO2 authentication token on an ESP32-S3 development board. The firmware presents a standard FIDO HID interface, implements the CTAP2 commands for information, credential creation and assertion, generates ECDSA P-256 keys on the device, stores credentials in flash with integrity checks and a persistent counter, and signs only after a button press. A relying party built with FastAPI and py_webauthn verifies every response.

On the hardware, a signature took 40.93 ms and key generation with its pairwise-consistency test 214.03 ms. The complete firmware occupies 344,752 bytes. On the host, the firmware sources passed more than 243,000 checks and 360,000 fuzzed inputs without a failure or crash, and every replay, wrong-origin, unknown-credential, missing-presence and counter-regression case was rejected. The prototype protects against remote attacks such as phishing, replay and database theft; without encrypted storage, secure boot and user verification it does not protect against an attacker who holds the device.

5.2 FUTURE ENHANCEMENTS

1. **Secure element.** A secure element would hold the private keys and perform the signatures; flash encryption and secure boot are an intermediate step.
2. **NFC transport.** An NFC controller and antenna would allow use with mobile phones; only a new transport module is required.
3. **Credential management utility.** A host application would list and delete stored credentials after authorisation.

REFERENCES

[1] J. Bonneau, C. Herley, P. C. van Oorschot and F. Stajano, "The quest to replace passwords: A framework for comparative evaluation of web authentication schemes," in *Proc. 2012 IEEE Symp. Security and Privacy*, 2012, pp. 553–567, doi: 10.1109/SP.2012.44.

[2] K. Thomas *et al.*, "Data breaches, phishing, or malware? Understanding the risks of stolen credentials," in *Proc. 2017 ACM SIGSAC Conf. Computer and Communications Security*, 2017, pp. 1421–1434, doi: 10.1145/3133956.3134067.

[3] P. A. Grassi *et al.*, "Digital identity guidelines: Authentication and lifecycle management," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST Special Publication 800-63B, Jun. 2017, doi: 10.6028/NIST.SP.800-63b.

[4] S. Srinivas, D. Balfanz, E. Tiffany and A. Czeskis, Eds., "Universal 2nd Factor (U2F) overview," FIDO Alliance Proposed Standard, Apr. 2017. [Online]. Available: https://fidoalliance.org/specs/fido-u2f-v1.2-ps-20170411/fido-u2f-overview-v1.2-ps-20170411.html (accessed Oct. 3, 2026).

[5] J. Hodges, J. C. Jones, M. B. Jones, A. Kumar and E. Lundberg, Eds., "Web Authentication: An API for accessing public key credentials, Level 2," W3C Recommendation, Apr. 2021. [Online]. Available: https://www.w3.org/TR/webauthn-2/ (accessed Oct. 3, 2026).

[6] FIDO Alliance, "Client to Authenticator Protocol (CTAP)," Proposed Standard, Jun. 2021. [Online]. Available: https://fidoalliance.org/specs/fido-v2.1-ps-20210615/fido-client-to-authenticator-protocol-v2.1-ps-20210615.html (accessed Oct. 3, 2026).

[7] J. Lang, A. Czeskis, D. Balfanz, M. Schilder and S. Srinivas, "Security keys: Practical cryptographic second factors for the modern web," in *Financial Cryptography and Data Security*, Lecture Notes in Computer Science, vol. 9603. Berlin, Germany: Springer, 2017, pp. 422–440, doi: 10.1007/978-3-662-54970-4_25.

[8] C. Bormann and P. Hoffman, "Concise Binary Object Representation (CBOR)," Internet Engineering Task Force, RFC 8949, Dec. 2020.

[9] J. Schaad, "CBOR Object Signing and Encryption (COSE): Structures and process," Internet Engineering Task Force, RFC 9052, Aug. 2022.

[10] National Institute of Standards and Technology, "Digital Signature Standard (DSS)," Federal Information Processing Standards Publication 186-5, Feb. 2023, doi: 10.6028/NIST.FIPS.186-5.

[11] M. Barbosa, A. Boldyreva, S. Chen and B. Warinschi, "Provable security analysis of FIDO2," in *Advances in Cryptology – CRYPTO 2021*. Cham, Switzerland: Springer, 2021, pp. 125–156, doi: 10.1007/978-3-030-84252-9_5.

[12] N. Bindel, C. Cremers and M. Zhao, "FIDO2, CTAP 2.1, and WebAuthn 2: Provable security and post-quantum instantiation," in *Proc. 2023 IEEE Symp. Security and Privacy*, 2023, pp. 1471–1490, doi: 10.1109/SP46215.2023.10179454.

[13] E. Ulqinaku, H. Assal, A. Abdou, S. Chiasson and S. Capkun, "Is real-time phishing eliminated with FIDO? Social engineering downgrade attacks against FIDO protocols," in *Proc. 30th USENIX Security Symp.*, Aug. 2021, pp. 3811–3828.

[14] S. G. Lyastani, M. Schilling, M. Neumayr, M. Backes and S. Bugiel, "Is FIDO2 the kingslayer of user authentication? A comparative usability study of FIDO2 passwordless authentication," in *Proc. 2020 IEEE Symp. Security and Privacy*, 2020, pp. 268–285, doi: 10.1109/SP40000.2020.00047.

[15] J. Reynolds, T. Smith, K. Reese, L. Dickinson, S. Ruoti and K. Seamons, "A tale of two studies: The best and worst of YubiKey usability," in *Proc. 2018 IEEE Symp. Security and Privacy*, 2018, pp. 872–888, doi: 10.1109/SP.2018.00067.

[16] E. Dauterman, H. Corrigan-Gibbs, D. Mazières, D. Boneh and D. Rizzo, "True2F: Backdoor-resistant authentication tokens," in *Proc. 2019 IEEE Symp. Security and Privacy*, 2019, pp. 398–416, doi: 10.1109/SP.2019.00048.

[17] T. Roche, V. Lomné, C. Mutschler and L. Imbert, "A side journey to Titan," in *Proc. 30th USENIX Security Symp.*, Aug. 2021, pp. 231–248.

[18] SoloKeys, "Solo 1: Open source FIDO2 security key." [Online]. Available: https://github.com/solokeys/solo1 (accessed Oct. 3, 2026).

[19] Google, "OpenSK: An open-source implementation for security keys written in Rust." [Online]. Available: https://github.com/google/OpenSK (accessed Oct. 3, 2026).

[20] Espressif Systems, "ESP-IDF programming guide for ESP32-S3, release v5.3." [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/v5.3/esp32s3/ (accessed Oct. 3, 2026).

[21] T. Pornin, "Deterministic usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA)," Internet Engineering Task Force, RFC 6979, Aug. 2013.

[22] E. Barker and J. Kelsey, "Recommendation for random number generation using deterministic random bit generators," National Institute of Standards and Technology, NIST Special Publication 800-90A Rev. 1, Jun. 2015, doi: 10.6028/NIST.SP.800-90Ar1.

APPENDIX I

RAW TIMING DATA

Table A1.1 Raw execution times of run 2 (optimised build), in microseconds

Iteration	Key generation + pairwise test	Pairwise test	Sign	Verify	Signature (bytes)
0	214078	178327	40930	137490	71
1	213883	178180	40882	137452	72
2	214172	178456	40894	137557	71
3	213901	178343	40856	137576	70
4	214243	178632	40952	137583	70
5	213930	178456	40844	137483	71
6	214124	178443	40912	137475	71
7	214009	178390	40959	137481	70
8	214127	178409	40914	137409	70
9	214094	178351	41018	137383	71
10	213966	178374	40959	137431	71
11	214027	178428	40944	137525	70
12	213942	178291	40943	137420	70
13	213975	178443	40971	137448	72
14	213836	178245	40969	137409	70
15	214138	178546	40923	137655	72
16	213993	178398	40866	137514	72
17	214043	178412	41009	137493	72
18	214008	178388	40903	137526	70
19	214119	178462	40898	137607	70

Table A1.2 Raw execution times of run 1 (unoptimised build, partial log), in microseconds

Iteration	Key generation + pairwise test	Sign	Verify	Signature (bytes)
16	720614	170016	327641	71
17	655959	170047	327210	72
18	656011	169803	327347	71
19	655823	169917	327629	71